

A Concise Derivation of t-SNE

Zhenwei Tang

Goal

Given n high-dimensional data points

$$x_i \in \mathbb{R}^p, \quad i = 1, \dots, n,$$

t-SNE (t-distributed Stochastic Neighbor Embedding) aims to find low-dimensional embeddings

$$y_i \in \mathbb{R}^d, \quad d \ll p \quad (\text{typically } d = 2 \text{ or } 3),$$

such that *neighborhood structure is preserved*: pairs of points that are close in the original space should also be close in the embedding, and pairs that are far apart should remain far apart.

The key idea is to turn pairwise distances into probability distributions both in the high-dimensional space and in the low-dimensional space, then minimize the discrepancy between these two distributions.

Step 1: Define Pairwise Similarities in High-Dimensional Space

Why: Raw Euclidean distances are hard to compare across different local densities. Converting them to conditional probabilities normalises for local density and gives a clear probabilistic interpretation of “neighbourhood”.

For each pair (i, j) , define the *conditional probability* that point x_i would pick x_j as its neighbour, modelled by a Gaussian centred at x_i :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}, \quad p_{i|i} = 0.$$

Here $\sigma_i > 0$ is a per-point bandwidth, chosen automatically by the **perplexity** criterion (see Appendix A).

To obtain a *symmetric* joint probability, average the two conditional probabilities:

$$\boxed{p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}}, \quad \sum_{i \neq j} p_{ij} = 1.$$

The symmetrisation ensures that $p_{ij} = p_{ji}$ and that every point has a meaningful influence on every other point.

Step 2: Define Pairwise Similarities in Low-Dimensional Space

Why: In the low-dimensional embedding we need a similarity measure that (a) is symmetric by construction and (b) has *heavier tails* than a Gaussian. Heavy tails are crucial: they allow moderately dissimilar points to be placed further apart without incurring a large penalty, alleviating the “crowding problem” (see Appendix B).

t-SNE uses a *Student’s t-distribution with one degree of freedom* (i.e. a Cauchy distribution) to define the joint similarity of the embeddings y_i, y_j :

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq \ell} (1 + \|y_k - y_\ell\|^2)^{-1}}, \quad q_{ii} = 0.$$

This is symmetric by construction: $q_{ij} = q_{ji}$.

Step 3: Formulate the Optimization Objective

Why: We want the distribution $Q = \{q_{ij}\}$ in embedding space to match the distribution $P = \{p_{ij}\}$ in the original space as closely as possible. The natural measure of how much one probability distribution diverges from another is the **Kullback–Leibler (KL) divergence** (see Appendix C).

The objective is:

$$\mathcal{L}(Y) = \text{KL}(P \parallel Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}},$$

where $Y = \{y_1, \dots, y_n\}$ collects all embedding coordinates.

Since P is fixed (computed once from the data), minimising $\mathcal{L}$ over Y is equivalent to maximising $\sum_{i \neq j} p_{ij} \log q_{ij}$, i.e. making Q as large as possible wherever P is large.

Step 4: Derive the Gradient

Why: $\mathcal{L}$ has no closed-form minimiser; we use **gradient descent** to iteratively update the embeddings. We therefore need $\partial \mathcal{L} / \partial y_i$.

Let $Z = \sum_{k \neq \ell} (1 + \|y_k - y_\ell\|^2)^{-1}$ denote the normalisation constant, so that

$$q_{ij} = \frac{w_{ij}}{Z}, \quad w_{ij} = (1 + \|y_i - y_j\|^2)^{-1}.$$

Expand the loss, dropping terms constant in Y :

$$\mathcal{L} = \sum_{i \neq j} p_{ij} \log p_{ij} - \sum_{i \neq j} p_{ij} \log q_{ij} = \text{const} - \sum_{i \neq j} p_{ij} (\log w_{ij} - \log Z).$$

Differentiating with respect to y_i (applying the chain rule):

Term 1: $\partial(\log w_{ij}) / \partial y_i$.

$$\frac{\partial \log w_{ij}}{\partial y_i} = \frac{1}{w_{ij}} \cdot \frac{\partial w_{ij}}{\partial y_i} = (1 + \|y_i - y_j\|^2) \cdot (-1)(1 + \|y_i - y_j\|^2)^{-2} \cdot 2(y_i - y_j) = -2 w_{ij} (y_i - y_j).$$

Term 2: $\partial \log Z / \partial y_i$.

$$\frac{\partial \log Z}{\partial y_i} = \frac{1}{Z} \sum_{k \neq \ell} \frac{\partial w_{k\ell}}{\partial y_i}.$$

Only terms with $k = i$ or $\ell = i$ are non-zero:

$$\frac{\partial \log Z}{\partial y_i} = \frac{1}{Z} \sum_{j \neq i} \left(\frac{\partial w_{ij}}{\partial y_i} + \frac{\partial w_{ji}}{\partial y_i} \right) = \frac{1}{Z} \sum_{j \neq i} 2 \cdot (-2 w_{ij}(y_i - y_j)) = -4 \sum_{j \neq i} q_{ij}(y_i - y_j).$$

(using $w_{ij} = w_{ji}$ and $q_{ij} = w_{ij}/Z$.)

Combining:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial y_i} &= - \sum_{j \neq i} p_{ij} \left[\frac{\partial \log w_{ij}}{\partial y_i} - \frac{\partial \log Z}{\partial y_i} \right] \\ &= - \sum_{j \neq i} p_{ij} \left[-2 w_{ij}(y_i - y_j) \right] + \sum_{j \neq i} p_{ij} \left[-4 \sum_{k \neq i} q_{ik}(y_i - y_k) \right]. \end{aligned}$$

Since $\sum_{j \neq i} p_{ij} = 1$ (after symmetrisation and normalisation is absorbed into P), the second term simplifies. Collecting and simplifying (using the symmetry $p_{ij} = p_{ji}$, $q_{ij} = q_{ji}$):

$$\boxed{\frac{\partial \mathcal{L}}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) w_{ij} (y_i - y_j)}.$$

Interpretation: Each pair (i, j) contributes an attractive force (when $p_{ij} > q_{ij}$, points are pulled together) or a repulsive force (when $p_{ij} < q_{ij}$, points are pushed apart), weighted by the heavy-tailed kernel w_{ij} .

Step 5: Optimization via Gradient Descent

The embeddings are updated iteratively. Using **gradient descent with momentum** (to accelerate convergence and reduce oscillations):

$$Y^{(t+1)} = Y^{(t)} - \eta \left. \frac{\partial \mathcal{L}}{\partial Y} \right|_{Y^{(t)}} + \alpha(t) (Y^{(t)} - Y^{(t-1)}),$$

where:

- $\eta > 0$ is the *learning rate*,
- $\alpha(t) \in (0, 1)$ is the *momentum coefficient* at step t ,
- $\partial \mathcal{L} / \partial Y$ stacks the per-point gradients from Step 4.

In practice, **early exaggeration** is applied during the first ~ 250 iterations: multiply all p_{ij} by a factor (e.g. 12) to encourage the formation of tight, well-separated clusters before fine-tuning.

Conclusion

t-SNE constructs a faithful low-dimensional embedding by:

1. Converting high-dimensional pairwise distances to a symmetric probability distribution P via per-point Gaussian kernels with perplexity-controlled bandwidths.
2. Defining a low-dimensional similarity distribution Q using a heavy-tailed Student- t kernel to counter the crowding problem.
3. Minimising the KL divergence $\text{KL}(P\|Q)$ via gradient descent, where the gradient admits the elegant closed form

$$\frac{\partial \mathcal{L}}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) w_{ij} (y_i - y_j).$$

t-SNE preserves local neighbourhood structure by matching a heavy-tailed low-dimensional similarity distribution to a Gaussian high-dimensional one, minimising their KL divergence through gradient descent.

Appendix A: Perplexity and Bandwidth Selection

The bandwidth σ_i for each point x_i is chosen so that the conditional distribution $P_{j|i}$ achieves a target *perplexity*:

$$\text{Perp}(P_i) = 2^{H(P_i)}, \quad H(P_i) = - \sum_{j \neq i} p_{j|i} \log_2 p_{j|i},$$

where $H(P_i)$ is the Shannon entropy of the conditional distribution. Perplexity can be interpreted as a smooth measure of the effective number of neighbours. A typical value is $\text{Perp} \in [5, 50]$. The bandwidth σ_i is found by binary search: increase σ_i if the current entropy is too low, decrease it if too high.

Appendix B: The Crowding Problem

In high dimensions, the volume of a sphere of radius r grows as r^p . When projecting onto $d \ll p$ dimensions, there is simply not enough area to faithfully represent all moderately distant neighbours—they get “crowded” into a small central region. Using a Gaussian in the low-dimensional space would require placing moderately similar points so close together that they overlap with genuinely similar ones.

The Student- t kernel $w_{ij} = (1 + \|y_i - y_j\|^2)^{-1}$ has a heavier tail than a Gaussian: for large $\|y_i - y_j\|$, w_{ij} decays as $\|y_i - y_j\|^{-2}$ rather than exponentially. This allows moderate distances in embedding space to correspond to moderate q_{ij} values, relieving the crowding.

Appendix C: Kullback–Leibler Divergence

For two discrete probability distributions $P = \{p_{ij}\}$ and $Q = \{q_{ij}\}$ over the same index set, the KL divergence is:

$$\text{KL}(P \| Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \geq 0,$$

with equality if and only if $P = Q$ (by Gibbs' inequality). KL divergence is *not* symmetric: $\text{KL}(P\|Q) \neq \text{KL}(Q\|P)$ in general.

In t-SNE, using $\text{KL}(P\|Q)$ (rather than $\text{KL}(Q\|P)$) has an important asymmetric effect:

- When p_{ij} is large but q_{ij} is small, the term $p_{ij} \log(p_{ij}/q_{ij})$ is very large $\rightarrow$ the loss strongly penalises placing nearby high-dimensional points far apart in the embedding.
- When p_{ij} is small but q_{ij} is large, the penalty is minimal $\rightarrow$ placing far-apart points nearby is less costly.

This asymmetry biases t-SNE towards faithfully representing local structure.

Appendix D: Gradient Descent with Momentum

Standard gradient descent updates parameters as $\theta \leftarrow \theta - \eta \nabla \mathcal{L}(\theta)$. **Momentum** adds a fraction α of the previous update:

$$v^{(t+1)} = \alpha v^{(t)} - \eta \nabla \mathcal{L}(\theta^{(t)}), \quad \theta^{(t+1)} = \theta^{(t)} + v^{(t+1)}.$$

Momentum damps oscillations in directions of high curvature and accelerates movement along consistent gradient directions, improving convergence speed and stability.